Power Transformer

Skewness, Box--Cox and Yeo--Johnson

Contents

Power Transformer 2
31.1 The problem a transform is meant to solve 2
31.2 Measuring skewness 3
31.3 The power family and the role of lambda 3
31.4 Box–Cox: definition and estimation 4
31.5 Yeo–Johnson: the same idea for all real values 5
31.6 The Scikit-learn workflow 6
31.7 What the lecture's concrete-strength experiment shows 7
31.8 Diagnostics, limitations and common mistakes 7
31.9 Recap, practice and sources 8

CHAPTER 31

Power Transformer

Skewness, Box–Cox and Yeo–Johnson

> **WHAT THIS CHAPTER BUILDS**
>
> A long tail is not a cosmetic problem: it changes the loss, the distances, and the meaning of an average. This chapter derives the power family, explains how lambda is chosen, and turns the derivation into a Scikit-learn workflow that is fitted on training data only.

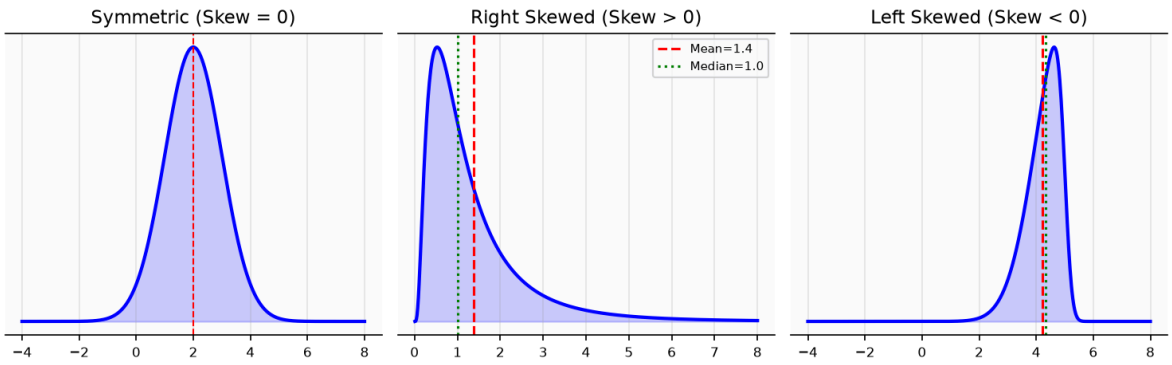
31.1 The problem a transform is meant to solve

Suppose a column contains ten incomes, in thousands:

Observation	Income
1	10
2	11
3	12
4	13
5	14
6	15
7	16
8	17
9	18
10	90

Nine observations sit between 10 and 18. One observation is 90. The mean is about 21.6, while the median is 14.5. The mean is not “wrong”; it is answering a different question. It is the balance point of the sample, and the single large value pulls that balance point to the right. A model that minimises squared error feels the same pull, because the residual of the 90 is squared.

The shape has a name. A distribution with a long right tail is **right-skewed** or **positively skewed**. A distribution with a long left tail is **left-skewed** or **negatively skewed**. A symmetric distribution has zero skewness. Figure 31.1 shows the three shapes.



Three distribution shapes. In the right-skewed case the mean is pulled toward the tail; in the left-skewed case it is pulled to the left.

Skewness matters differently to different models. Linear regression with squared loss gives extreme observations quadratic influence. Distance-based methods such as k-nearest neighbours, k-means, and support-vector machines compare coordinates directly, so a feature with a long tail can dominate the distance. Tree-based models split on order, not on the raw spacing, and are therefore less sensitive; they are not immune, because the transformed feature can still produce more useful split points.

A transform changes leverage, not truth

A power transform is a monotone change of scale. It does not delete an outlier or create information. It changes how much influence the tail has on the loss and on distances. That is why the same transform can help a linear model and do almost nothing for a tree.

31.2 Measuring skewness

For a random variable X with mean μ and standard deviation σ, the population skewness is the third central moment divided by σ^3:

$$\operatorname{Skewness}(X) = \frac{E\left[(X-\mu)^3\right]}{\sigma^3}$$

Dividing by σ^3 makes the number dimensionless, so skewness can be compared across features measured in different units. For a sample of n observations, the Fisher–Pearson coefficient is

$$g_1 = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i-\overline{x})^3}{\left(\frac{1}{n}\sum_{i=1}^{n}(x_i-\overline{x})^2\right)^{\frac{3}{2}}}$$

The sign is the direction of the tail: positive means the right tail is longer; negative means the left tail is longer. The magnitude is a rough severity score. A common convention treats $|g_1| < 0.5$ as approximately symmetric, $0.5 \le |g_1| < 1$ as moderate, and $|g_1| \ge 1$ as strong skewness. These are conventions, not laws; a feature with skewness 0.7 may still be perfectly usable.

There is a useful diagnostic relationship: in a right-skewed distribution the mean is usually greater than the median; in a left-skewed distribution the mean is usually less than the median. The reverse is not a proof of skewness, because a multimodal distribution can have mean equal to median and still be asymmetric. Always look at a histogram or density plot, not only at one number.

Which skewness does pandas report?

`Series.skew()` returns an unbiased estimator based on the adjusted Fisher–Pearson coefficient. It is not exactly the population formula above, and it can be noisy for small samples. Use it as a screening statistic, then inspect the distribution.

31.3 The power family and the role of lambda

The Box–Cox and Yeo–Johnson transforms belong to a one-parameter family. For positive x, the core expression is

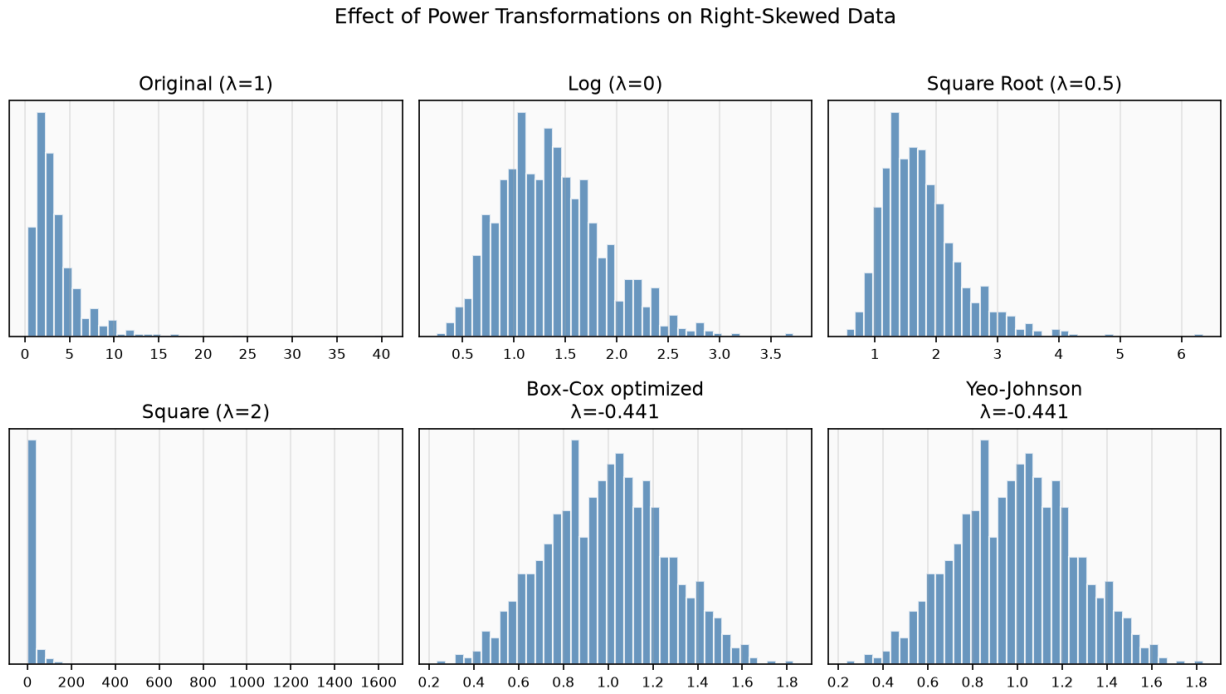
$$x^{(\lambda)} = \frac{x^{\lambda}-1}{\lambda}, \quad \lambda \neq 0$$

The parameter λ controls the curvature. When $\lambda = 1$ the expression is $x - 1$, a shift, so the shape does not change. When $\lambda = 0.5$ it is proportional to $\sqrt{x}$, which compresses large values. When $\lambda = -1$ it is $1 -$

$\frac{1}{x}$, which compresses large values much more aggressively. When $\lambda = 0$ the expression is undefined as written, but the limit exists:

$$\lim_{\lambda \to 0} \frac{x^{\lambda} - 1}{\lambda} = \ln x$$

This is the reason the log transform is not a separate trick. It is the continuous extension of the power family at $\lambda = 0$. Figure 31.2 shows the same right-skewed data after several values of λ.



The same data under different power transforms. A value of lambda near 0 produces a log-like compression; a positive value near 1 leaves the shape almost unchanged; a negative value over-compresses and can create a new spike near zero.

Lambda	Transform	Typical effect on a right tail
1	Identity (shifted)	Almost no change
0.5	Square root	Moderate compression
0	Logarithm	Strong compression of large values
-1	Reciprocal	Very strong compression; may create a pile-up near zero
2	Square (if all values positive)	Expands the tail; usually the wrong direction

The transform must preserve order to be safe in a supervised pipeline. For positive x and the Box–Cox form, the derivative is $x^{\lambda-1}$. For $\lambda > 0$ it is positive; for $\lambda < 0$ it is negative but never zero. In either case the function is strictly monotone, so the largest observation stays the largest and the smallest stays the smallest. Ranks, and therefore tree splits, are unchanged. The numerical spacing, and therefore the loss and the distances, are changed.

31.4 Box–Cox: definition and estimation

The Box–Cox transform is

$$y^{(\lambda)} = \begin{cases} \frac{x^{\lambda}-1}{\lambda} & \text{if } \lambda \neq 0 \\ \ln x & \text{if } \lambda = 0 \end{cases}$$

It requires every value to be strictly positive. Zero is not allowed, and a negative value is not allowed. If a feature contains zeros, one option is to add a small constant before transforming; that changes the interpretation and must be recorded. A better option, when zeros or negatives are genuine, is Yeo–Johnson.

The parameter λ is not chosen by hand in practice. It is estimated by maximum likelihood under the working assumption that the transformed data are approximately normal. For a sample $x_1, \dots, x_n$, the Box–Cox log-likelihood can be written as

$$\ell(\lambda) = -\frac{n}{2}\ln(2\pi) - \frac{n}{2}\ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}\left(y_i^{(\lambda)} - \overline{y}^{(\lambda)}\right)^2 + (\lambda - 1)\sum_{i=1}^{n}\ln x_i$$

The first three terms measure how well a normal distribution fits the transformed values. The final term is the Jacobian of the transformation; without it, the likelihood would compare densities on different scales and would be meaningless. Maximising $\ell(\lambda)$ over a grid or with a numerical optimiser gives the estimate. `scipy.stats.boxcox` returns both the transformed array and the estimated λ; `sklearn.preprocessing.PowerTransformer` stores the estimate in `lambdas_`.

A small numerical example makes the likelihood idea concrete. Take $x = (1, 2, 3, 4, 10)$. This sample is right-skewed: most values are small, one is large. A grid search over λ from -2 to 2 finds a negative value, because a negative power compresses the 10 toward the rest. If we force $\lambda = 0$, we get the log transform; if we force $\lambda = 1$, nothing changes. The fitted value is the one that makes the transformed sample closest to normal **and** accounts for the Jacobian. The result is not a universal constant; it depends on the sample.

> **Reading a fitted lambda**
>
> A fitted λ near 1 means the data already looked roughly normal on the original scale. A fitted λ near 0 means a log-like transform was preferred. A fitted λ near 0.5 means a square-root-like compression was preferred. A negative λ means the tail was strong enough that the optimiser wanted to compress it more than a log does.

Once a transform is fitted, it can be inverted. For $\lambda \neq 0$,

$$x = \left(\lambda y^{(\lambda)} + 1\right)^{\frac{1}{\lambda}}$$

and for $\lambda = 0$, $x = \exp\left(y^{(\lambda)}\right)$. The inverse matters when a model predicts in transformed space but the business needs the original units. Scikit-learn provides `inverse_transform` for this purpose.

31.5 Yeo–Johnson: the same idea for all real values

Yeo–Johnson modifies the power family so that zero and negative values are legal. The definition has four branches:

$$y^{(\lambda)} = \begin{cases} \frac{(x+1)^{\lambda}-1}{\lambda} & \text{if } x \geq 0, \lambda \neq 0 \\ \ln(x+1) & \text{if } x \geq 0, \lambda = 0 \\ \frac{1-(1-x)^{2-\lambda}}{2-\lambda} & \text{if } x < 0, \lambda \neq 2 \\ -\ln(1-x) & \text{if } x < 0, \lambda = 2 \end{cases}$$

The two branches for $x \geq 0$ are exactly Box–Cox applied to $x + 1$. The two branches for $x < 0$ mirror the construction so that the function and its first derivative match at zero. That continuity is the reason the transform is defined for the whole real line. The price is a more complicated derivative and a slightly less transparent interpretation of λ.

The important practical difference is coverage, not magic. If a feature is strictly positive and the fitted Box–Cox λ is stable, the two methods often produce similar results. If a feature contains zeros or negatives, Box–Cox is simply unavailable and Yeo–Johnson is the appropriate member of the family.

Property	Box–Cox	Yeo–Johnson
Allowed values	Strictly positive	All real values
Parameter	One lambda	One lambda, with a branch at zero
Typical use	Positive measurements: income, price, size	Any real feature, especially one with zeros or negatives
Failure mode	Raises an error on non-positive data	No domain error; still can over-transform
Scikit-learn	`method="box-cox"`	`method="yeo-johnson"` (default)

31.6 The Scikit-learn workflow

`PowerTransformer` implements both methods. Its important parameters are `method`, `standardize`, and `copy`. The default method is `"yeo-johnson"`; `standardize=True` applies zero-mean, unit-variance scaling after the power transform. That default is easy to miss: the output is not only the power-transformed value, it is the standardized power-transformed value. If you need the raw transformed value, set `standardize=False`.

The object learns one λ per feature during `fit`. After fitting, `lambdas_` has one entry per input column. `transform` applies the learned values; `inverse_transform` reverses both the standardization and the power step.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PowerTransformer

# X is a DataFrame of numeric features; y is the target.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

pt = PowerTransformer(method="yeo-johnson", standardize=True)
X_train_pt = pt.fit_transform(X_train)
X_test_pt = pt.transform(X_test)

print(pt.lambdas_)              # one fitted lambda per column
print(pt.feature_names_in_)     # columns seen during fit
```

The split is not decoration. The lambdas are estimated parameters. If you fit the transformer on the full dataset before splitting, information from the test rows has already influenced the transform. The result is data leakage, and the test score becomes optimistic. Fit on `X_train`, then transform `X_test` with the same object. This is the same rule as scaling, encoding, and imputation.

A complete pipeline keeps the rule enforceable:

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression

pipe = Pipeline([
```

```
    ("power", PowerTransformer(method="yeo-johnson")),
    ("model", LinearRegression()),
])
pipe.fit(X_train, y_train)
score = pipe.score(X_test, y_test)
```

For Box–Cox, the positivity requirement must be checked before the pipeline is fitted. A column with a zero will raise an error inside `fit`, and the traceback points at the transformer rather than at the offending value. Check `(X_train <= 0).sum()` first, then decide whether to shift the feature, drop it, or switch to Yeo–Johnson.

> **Do not transform the target by reflex**
>
> `PowerTransformer` is for features. Transforming the target changes the loss and the interpretation of every prediction. If you do transform y, you must invert the prediction before reporting an error in the original units, and you must check whether the model's error assumptions still make sense. The lecture's demonstration focuses on features; keep that scope unless you have a reason to go further.

31.7 What the lecture's concrete-strength experiment shows

The lecture uses a concrete compressive-strength dataset. The inputs describe the mixture (cement, blast-furnace slag, fly ash, water, superplasticiser, coarse aggregate, fine aggregate, and age); the target is compressive strength. Several input columns are strongly right-skewed, which makes the dataset a useful stress test for a linear model.

The workflow is deliberately simple: split the data, fit a linear regression, record R^2, then repeat after a power transform. In the lecture the untransformed baseline is around 0.62, and the transformed models are also around 0.62 to 0.63. The improvement is real but small. That is an honest result, and it is more instructive than a fabricated dramatic gain. A transform can only help if the model's assumptions are actually being violated in a way the transform can repair.

The lecture also checks the distributions before and after. Cement, blast-furnace slag, and superplasticiser are visibly skewed; after Yeo–Johnson they are closer to symmetric. Water and the aggregates are already closer to symmetric and change less. This is the correct diagnostic order: look at the feature, fit the transform, look at the feature again, and only then compare model performance.

> **Why the gain can be small**
>
> A linear model can fit a curved relationship poorly even after a marginal transform, because the transform is applied feature by feature. If the true relationship involves interactions or a non-linear link, a power transform is not a substitute for a better model. It is one repair for one assumption, not a general performance switch.

31.8 Diagnostics, limitations and common mistakes

The first diagnostic is the shape of the feature, not the model score. Plot a histogram or kernel density estimate before and after. A good transform reduces the long tail and moves the mean toward the median. A bad transform creates a new spike at the boundary, which is common with an aggressive negative λ on data that contain a natural floor.

The second diagnostic is the fitted λ. A value far outside the usual range, or a value that changes wildly when a few rows are removed, is a sign that the sample is small or the tail is dominated by a few points. In that case the transform is unstable, and a simple log or square root may be more defensible than a fitted parameter.

The third diagnostic is the model's assumptions. Power transforms are often described as "making data normal", but a feature transform does not make the residuals normal, does not fix heteroscedasticity by itself, and does not make a non-linear relationship linear. Check residual plots and cross-validation, not only the skewness of the input.

Common mistakes:

- Fitting the transformer on the full dataset before the train–test split. This leaks test information.
- Applying Box–Cox to a column with zeros or negatives. The transform is undefined; use Yeo–Johnson or a documented shift.
- Forgetting that `standardize=True` is the default. The output is standardized, not just power-transformed.
- Transforming every numeric column automatically. A feature that is already symmetric may not benefit, and a tree model may not need the change.
- Interpreting a transformed coefficient as if it were on the original scale. The coefficient belongs to the transformed feature; invert predictions, not coefficients, when communicating results.
- Using the transform on the target without inverting predictions before computing business metrics.

31.9 Recap, practice and sources

The chain to remember is: a long tail gives a few observations disproportionate leverage; a power transform changes the spacing while preserving order; λ controls the curvature; Box–Cox covers positive data; Yeo–Johnson covers all real data; Scikit-learn estimates one λ per feature and standardizes by default; fit on training data and transform the test set with the same object.

> **Lock-in line**
>
> A power transform does not remove a tail. It changes how much the tail is allowed to pull the loss and the distances.

Practice

1. Generate 1,000 samples from a log-normal distribution. Compute skewness before and after `PowerTransformer(method="yeo-johnson")`. Which λ is fitted?
2. Add a single extreme value to a symmetric sample. Recompute skewness and the fitted λ. How stable is the estimate?
3. Take a column with one zero. Show the exact error from Box–Cox, then fix it with Yeo–Johnson.
4. Compare `standardize=True` and `standardize=False` on the same fitted transformer. Verify the relationship between the two outputs.
5. Explain why a tree model may show almost no change after a power transform even when the histogram changes dramatically.

Sources and scope

Video 31, "Power Transformer", CampusX 100 Days of Machine Learning. The lecture covers skewness, Box–Cox, Yeo–Johnson, the concrete-strength experiment, and the Scikit-learn implementation. The derivation of the profile likelihood and the train–test leakage warning are supplementary explanations added here to make the mechanism complete; they are consistent with the lecture's implementation but are not spoken verbatim in it.